

A Formal Compatibility Checker for OpenAPI Specifications

Dakshesh Pandey (2417828)
University of Birmingham
2025-26

Abstract

Changes to web APIs can easily introduce compatibility problems for existing clients, especially when compatibility between versions is checked only through manual review or testing. This project develops a compatibility checker for OpenAPI specifications based on a formally defined notion of when one API version can safely replace another. The compatibility checks are implemented in Agda, while additional components translate OpenAPI specifications and provide a backend and web interface for running the analysis. When compatibility fails, the system reports structured information describing the source of the incompatibility.

1 Introduction

Web APIs are a central part of how modern software systems communicate. As APIs evolve, changes to endpoints, parameters, or response formats can unintentionally break existing clients. Even small modifications can introduce subtle incompatibilities that lead to failures in client applications [1]. Backward compatibility is widely recognised as a governance requirement for API providers [2], yet in practice it is still often assessed through manual review or informal testing, which offers limited guarantees as systems grow.

This project explores a more principled approach by modelling API compatibility as a refinement problem. Informally, refinement means that every interaction permitted by the old API remains valid in the new one. For example, a response schema may be extended with additional optional fields, but removing fields that existing clients expect would break compatibility. The goal is to build a compatibility checker that evaluates compatibility against this refinement relation and provides structured explanations when compatibility fails. A key challenge is connecting a formal, proof-oriented core with developer-facing tooling. The formal semantics and decision procedures are implemented in Agda, while Python, FastAPI, and React provide translation, execution, and visualisation layers around the verified core.

This approach also raises challenges, both in the formal development and in integrating the checker into a usable end-to-end pipeline. These are discussed in Sections 4 and 5.

1.1 Project Objectives

The objectives of the project are:

1. Define a formal model of API well-formedness and refinement using Agda.
2. Implement executable decision procedures for well-formedness and compatibility.
3. Translate OpenAPI specifications into the internal DSL used by the checker.
4. Provide developer-facing tooling for running and visualising compatibility checks.

2 Background

2.1 API Compatibility

Web APIs change over time as systems add features, modify data formats, or adjust endpoint behaviour. The challenge is determining whether these changes remain compatible with clients that were written against earlier versions of the API [1]. In this project, compatibility is treated as a *directional* relation from the old API to the new API. In other words, the question is whether the newer API version can safely replace the older one for existing clients without breaking previously valid interactions.

Some changes preserve compatibility, while others introduce *API drift*: structural differences between versions that may break previously valid client interactions. For example, adding optional information to a response may be harmless, whereas removing fields that clients rely on may cause failures. Similarly, introducing stricter requirements on request inputs can invalidate previously valid client interactions. Table 1 shows a few common examples.

Change	Compatible?
Add an optional field to a response schema	Yes
Add a new endpoint	Yes
Remove a field from a response schema that clients rely on	No
Introduce a new required request parameter	No

Table 1: Examples of API changes and their compatibility

Rather than treating compatibility as only a yes/no property, it is useful to identify the specific structural change responsible for incompatibility. In this work, such evidence is represented as a *drift witness*.

Example Consider a simple API with a single endpoint `GET /users` that returns objects with fields `id` and `name`, as shown in Listing 1. If a new version adds an optional field `email`, existing clients continue to work unchanged. Removing the `name` field, however, would break clients that rely on it. This captures the directional notion of compatibility used here.

Listing 1: Simplified OpenAPI example

```
openapi: 3.1.0
paths:
  /users:
    get:
      responses:
        "200":
          description: ok
          content:
            application/json:
              schema:
                type: object
                properties:
                  id:
                    type: string
                  name:
                    type: string
```

2.2 OpenAPI Specifications

OpenAPI is a widely used standard for describing HTTP APIs [3]. An OpenAPI specification defines the structure of an API in terms of endpoints, HTTP methods, parameters, request bodies, and response schemas. These specifications provide a structured description of how clients interact with an API.

In this project, OpenAPI specifications serve as the input representation for APIs. However, rather than modelling the entire OpenAPI standard, we focus on a restricted subset that captures the structural features needed for formal reasoning about API evolution.

Restricting the language serves two purposes. First, it keeps the formal model manageable by focusing on the core constructs of the subset, including schemas, endpoints, parameters, and responses. Second, it ensures that specifications can be translated into the formal representation deterministically and without ambiguity.

More dynamic or complex features of OpenAPI, such as flexible schema composition or constructs whose behaviour depends on runtime context, are not included. These features are difficult to model precisely in a static setting and would significantly increase the complexity of the formal development.

A more detailed description of the supported subset and design decisions is available in the project repository ([docs/exploringOpenAPI.md](#)).

3 System Architecture

The architecture follows a layered design centred around the Agda formalisation, which defines the internal representation of APIs, the well-formedness conditions that specifications must satisfy, and the compatibility relation used to compare API versions. Surrounding components translate OpenAPI specifications into this representation and expose the analysis through programmatic and user-facing interfaces.

The system is organised into four layers:

- 1. Formal core (Agda).** The Agda layer defines the internal API model together with the rules for well-formedness, refinement, and compatibility checking. This layer acts as the semantic authority for compatibility outcomes.
- 2. Translation layer (Python).** A Python parser reads OpenAPI YAML specifications, checks that they fall within the supported subset, and translates them into the internal representation used by the Agda model.
- 3. Backend service (FastAPI).** The backend exposes compatibility checking through asynchronous job endpoints and manages execution stages, logs, and structured result output.
- 4. Frontend interface (React).** The frontend provides an interface for submitting API specifications and inspecting compatibility results and diagnostic information.

This separation keeps the formal reasoning in Agda independent of the surrounding implementation layers. The Agda model determines compatibility, while the other components handle specification translation, execution, and presentation. Figure 1 summarises the system architecture and runtime flow.

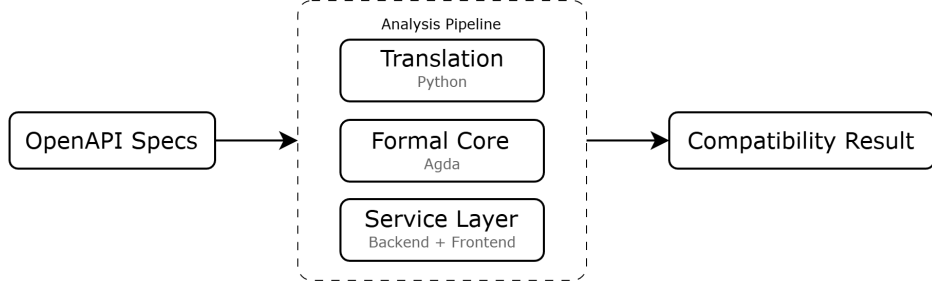


Figure 1: System architecture and runtime flow

4 Formal Model of API Compatibility

The formal compatibility model is implemented in Agda, a dependently typed programming language and proof assistant [4]. The Agda development forms the formal core of the system, defining the internal API representation, well-formedness conditions, and the refinement relation used to compare API versions. The surrounding components described in Section 3 translate OpenAPI specifications into this representation and call the Agda checker to compute compatibility results.

4.1 API Representation

APIs are represented using a domain-specific language. The syntax mirrors the structure of OpenAPI specifications, restricting the language to the subset required for compatibility analysis. The DSL is organised into four layers, summarised in Table 2.

DSL Layer	Contents	OpenAPI Section
HTTP Primitives	Methods and status codes	Core HTTP concepts used throughout the specification
Data Level	Schemas and base types	<code>components/schemas</code>
Operation Level	Parameters, paths, request bodies, responses, endpoints	<code>paths</code> and their nested operations
API Level	Top-level API definition and reusable schemas	Root OpenAPI document (<code>info</code> , <code>paths</code> , <code>components</code>)

Table 2: Structure of the internal DSL.

The HTTP layer introduces the basic protocol primitives used across the specification, such as HTTP methods and response status codes. These provide the constants used by higher levels of the DSL.

The data layer defines the structure of information exchanged between clients and servers. A `Schema` describes the shape of a request or response payload. As shown in Listing 2, it includes a base `type` (such as `object` or `string`), a list of `properties` mapping field names to nested schemas, a `required` list identifying mandatory fields, and an optional `items` field for array types.

At the operation level, an `Endpoint` models a single API operation, as shown in Listing 3. It records the `route` and HTTP `method` that identify the operation, the `parameters` it accepts

(such as path or query parameters), the optional request **body**, and the list of possible **responses**.

Listing 2: Schema definition (⚙️)

```
record Schema : Set where
  inductive
  field
    type      : Base
    properties : List (String × Schema)
    required  : List String
    items     : Maybe Schema
```

Listing 3: Endpoint definition (⚙️)

```
record Endpoint : Set where
  field
    route      : Path
    method     : Method
    parameters : List Parameter
    body       : Body method
    responses  : List Response
```

Finally, the API level aggregates the full specification. As shown in Listing 4, an API record combines a list of **paths** (the defined endpoints) with a list of named **components**, which are reusable schema definitions that endpoints may reference.

Listing 4: API definition (⚙️)

```
record API : Set where
  field
    paths      : List Endpoint
    components : List (String × Schema)
```

These definitions provide the internal representation used throughout the model. Since the syntax alone permits structurally invalid specifications, well-formedness conditions are introduced to define the valid subset.

4.2 Well-Formed Specifications

Well-formedness predicates restrict the DSL syntax to API descriptions that satisfy the structural constraints of the model.

Listing 5 shows the rule for well-formed object schemas. A value of type **WFSchema** **s** is a proof that the schema **s** is well-formed. The constructor **wf-object** requires six conditions to hold. The schema's **type** must be **object** and its **items** field must be **nothing**, so it does not also behave as an array. The **All WFSchema** condition checks that every nested property schema is itself well-formed. The second **All** constraint ensures that every required field name appears in **properties**. Finally, the two **Unique** conditions rule out duplicate property names and duplicate required entries.

Listing 5: Well-formedness rule for object schemas (⚙️)

```
data WFSchema : Schema → Set where
  wf-object :
    ∀ {s}
    → Schema.type s ≡ object
    → Schema.items s ≡ nothing
    → All WFSchema (values (Schema.properties s))
    → All (λ k → k ∈ keys (Schema.properties s)) (Schema.required s)
    → Unique (keys (Schema.properties s))
    → Unique (Schema.required s)
    → WFSchema s
```

For example, a schema that marks a field as **required** without declaring it in **properties** is rejected. Similar predicates are defined for endpoints and APIs, ensuring that compatibility checking is performed only on structurally valid specifications.

4.3 Compatibility as Refinement

Compatibility between API versions is defined as a refinement relation, compositional over schemas, endpoints, and complete APIs.

At the data level, schema refinement captures when one schema can safely replace another. This follows standard variance principles from type theory [5]. For request inputs, the newer API must accept everything that was previously valid; otherwise existing clients could send requests that are no longer supported. For responses, the newer API may return additional information, but it must preserve the fields that clients already depend on. Requiring exact schema equality would reject safe evolutions like adding optional fields, so a variance-aware approach is needed to capture what remains backwards compatible.

Listing 6 shows the object schema refinement rule. A value of type `Schema ⊆ Co s t` represents a proof that the newer schema `t` safely refines the original schema `s`. The constructor `⊆-object` requires both schemas to be well-formed objects. The `PropsRefine` condition checks that every property in `s` appears in `t` with a compatible type, while allowing `t` to introduce additional optional properties. The final condition requires that the required fields of `s` are a subset of those in `t`, so no previously required field is dropped.

Listing 6: Object schema refinement rule (⚙️)

```
data Schema ⊆ Co : Schema → Schema → Set where
  ⊆-object :
    ∀ {s t}
      → WFSchema s
      → WFSchema t
      → Schema.type s ≡ object
      → Schema.type t ≡ object
      → PropsRefine Schema ⊆ Co (Schema.properties s) (Schema.properties t)
      → Schema.required s ⊆ Schema.required t
      → Schema ⊆ Co s t
```

These rules lift naturally to higher levels of the model. Endpoint refinement compares operations with the same route and HTTP method, checking that parameters, request bodies, and responses evolve compatibly. At the top level, API refinement requires every endpoint and component schema in the original API to be present and refined in the newer version.

Challenge 1: Refinement as a preorder. A key difficulty in the formalisation is showing that refinement forms a preorder, requiring proofs of reflexivity and transitivity, established in the repository (⚙️). These proofs require a number of supporting lemmas before the main results can be stated. The object case is particularly tricky, since property lists behave as finite maps and refinement witnesses must be carried across intermediate lookup results. Transitivity is more involved. An earlier version factored part of the object proof into a separate `PropsRefine-trans` helper that took the inductive hypothesis as an argument. Although the recursion was valid, this obscured the structural decrease from Agda’s termination checker [6] and required a `{-# TERMINATING #-}` annotation. Inlining that reasoning directly into the object case via a `where` clause makes the decrease visible, removing the need for the annotation.

4.4 Drift Witnesses

When compatibility fails, the model produces a *drift witness* describing the structural change responsible for the incompatibility. Rather than returning only a Boolean result, the checker shows exactly how the newer specification differs from the original.

The `Drift` type is indexed by two API values, `a0` (the original) and `a1` (the newer version).

Its constructors classify incompatibilities at two levels of the API structure. At the schema level, `ComponentRemoved` and `ComponentDriftWitness` describe changes to components. At the endpoint level, `EndpointRemoved` and `EndpointDriftWitness` capture changes to endpoints, as shown in Listing 7.

Listing 7: Drift constructors (⚙️)

```
data Drift : API → API → Set where
  ComponentRemoved :
    ...

  ComponentDriftWitness :
    ...

  EndpointRemoved :
    lookupEndpoint r m (API.paths a0) ≠ nothing →
    lookupEndpoint r m (API.paths a1) ≡ nothing →
    Drift a0 a1

  EndpointDriftWitness :
    ...
```

As a concrete example, `EndpointRemoved` takes two proofs as arguments. The first shows that looking up route `r` with method `m` in `a0` does not return `nothing`, so the endpoint exists in the original API. The second shows that the same lookup in `a1` returns `nothing`, so it has been removed in the newer version. Together, these provide evidence that a breaking removal has occurred. The schema-level constructors follow the same idea, but operate over `API.components` rather than `API.paths`.

Drift witnesses are both *sound* and *complete* with respect to the refinement relation, meaning that a witness is produced if and only if refinement does not hold (⚙️). This ensures that the checker does not silently miss a breaking change.

4.5 Decidable Compatibility Checking

The predicates introduced above are accompanied by decision procedures that allow compatibility checks to be executed automatically. Each procedure either constructs a proof that a property holds, or returns evidence explaining why it fails.

Listing 8: Decision procedures for well-formedness and compatibility

```
WFAPI? : (api : API) → WFAPI api ⊢ APIIllFormed api

API⊑? : (a0 a1 : API) → WFAPI a0 → WFAPI a1 → API⊑ a0 a1 ⊢ Drift a0 a1
```

Both procedures return a value of a sum type, written $\dot{+}$, whose two constructors represent the possible outcomes. `WFAPI?` takes a single API and returns either a proof of `WFAPI`, confirming that the specification is structurally valid, or an `APIIllFormed` value identifying the violation (⚙️). `API⊑?` takes two APIs together with proofs that both are well-formed, and returns either a refinement proof `API⊑ a0 a1` confirming compatibility, or a `Drift` witness describing the breaking change (⚙️). Well-formedness is required as a precondition to ensure that compatibility is only checked on valid inputs.

This makes the formal definitions executable while still producing structured evidence in both success and failure cases.

Challenge 2: Returning useful error information. A key difficulty in the implementation was moving beyond a simple yes/no compatibility check. An initial version used `Dec (API $\sqsubseteq$ )`, returning only `yes` or `no`. While the negative case gives a proof that refinement does not hold, it gives no indication of what actually caused the failure.

To address this, the procedure was changed to return `API $\sqsubseteq$  a0 a1  $\dot{+}$  Drift a0 a1`, so that failures produce a concrete `Drift` witness describing the incompatibility. This was also important for the proof development. The harder direction is showing that every refinement failure can be explained by some drift witness. At a high level, the proof uses the fact that if `P  $\dot{+}$  Q` and $\neg P$, then `Q`. Here, `P` is `API $\sqsubseteq$  a0 a1` and `Q` is `Drift a0 a1`. Since `API $\sqsubseteq$ ?` returns one or the other, assuming refinement does not hold rules out the first case and leaves a concrete drift witness. This gives the completeness direction for drift.

5 Tooling and Execution

Section 4 defines compatibility over an internal API representation. This section describes how that model is used in practice. OpenAPI specifications are translated into the DSL, checked using the Agda implementation, and the results are exposed through backend and frontend components.

5.1 Translation and Execution

OpenAPI specifications are provided in YAML and translated into a representation suitable for formal checking. This translation is implemented in Python ([parser/](#)) and acts as the bridge between flexible input documents and the strictly typed Agda core.

Specifications are first parsed from YAML and checked for basic structure, such as the presence of `openapi` and `paths`. This catches malformed inputs early and prevents invalid data from reaching later stages.

The main translation step converts the parsed dictionary into an internal representation defined using Python dataclasses. This representation mirrors the DSL used in the Agda formalisation, capturing APIs in terms of schemas, endpoints, parameters, and responses. For instance, paths are decomposed into literal and parameter segments, and HTTP methods are normalised to a fixed set.

At this point, the OpenAPI subset is structurally constrained: parameters are limited to primitive types and must appear in either `path` or `query`; request and response bodies are JSON; and schema references resolve through `components/schemas`. This keeps the specification within a deterministic fragment that the formal model can represent, while semantic well-formedness checks such as required path parameters and duplicate detection are deferred to the Agda core.

A simplified example of this translation is shown below.

Listing 9: Input (OpenAPI YAML)

```
paths:
  /users:
    get:
      responses:
        "200":
          content:
            application/json:
              schema:
                type: object
```

Listing 10: Generated API representation (Agda)

```

Path0 : Path
Path0 = record { segments = lit "users" :: [] }

Responses0 : List Response
Responses0 = response OK record { type = object ; properties = [] ; required = [] ;
    items = nothing } :: []

Endpoint0 : Endpoint
Endpoint0 = record { route = Path0 ; method = GET ; parameters = [] ; body = NoBody ;
    responses = Responses0 }

GeneratedAPI : API
GeneratedAPI = record { paths = Endpoint0 :: [] ; components = [] }

```

The validated representation is then rendered into executable Agda modules. For each API, the generator produces a module that constructs the specification using the DSL from Section 4, together with a small runner that invokes well-formedness and compatibility checks.

These modules are compiled using Agda’s Haskell backend into standalone executables, which are then run from the Python layer. The checker first evaluates well-formedness for each API and, if both are valid, performs compatibility checking in the direction *old* $\rightarrow$ *new* using the decision procedure from Section 4.5.

The result of execution is captured and mapped to a structured output. Successful checks produce a compatibility result, while failures return a drift witness describing the source of the incompatibility. This output is then passed to the backend for processing and presentation.

Challenge 3: Executing the formal checker in practice. A practical challenge was integrating the Agda formalisation into the tool pipeline. For each pair of specifications, the Python layer generates Agda runner modules that invoke the decision procedures directly, then compiles them via Agda’s GHC backend into standalone executables. Results are communicated back to Python through structured plain-text output on `stdout`, with tagged lines indicating success or failure and key-value pairs providing context about the incompatibility. While this keeps execution faithful to the formal definitions, it introduces overhead from per-run compilation and requires a carefully maintained interface between the generated Agda code and the surrounding Python system.

5.2 Backend Service (FastAPI)

The backend handles execution of compatibility checks and manages their lifecycle. It is implemented using FastAPI ([apps/backend/](#)) and exposes endpoints for submitting specifications, tracking progress, and retrieving results.

Compatibility checks are executed as asynchronous jobs. When a request is submitted, the backend assigns it a unique identifier and processes it through the translation and execution pipeline described earlier. This staged design makes it possible to localise failures to specific phases, such as translation errors or compatibility violations.

Throughout execution, logs and intermediate status updates are captured and made available to the client. This allows users to monitor progress and inspect failures without direct access to the underlying system.

Once execution completes, the backend returns a structured result indicating compatibility status and any drift information, which is consumed directly by the frontend.

5.3 Frontend Interface (React)

The frontend provides an interface for submitting specifications and visualising compatibility results. It is implemented as a web application using React and TypeScript ([apps/frontend/](#)).

Users provide two OpenAPI specifications through text-based input panels. These are sent to the backend, which initiates a compatibility check and returns a job identifier. The frontend follows an asynchronous polling model, periodically querying the backend for updates until execution completes.

During execution, the interface displays progress information, including elapsed time and intermediate status updates. This keeps long-running checks transparent and the interface responsive.

Once a result is available, it is rendered in a structured format. Successful checks are reported directly, while incompatibilities are presented with descriptive messages derived from structured backend tags. Context such as affected endpoints or parameters is prioritised to highlight the source of the issue.

The frontend also supports interaction features such as copying results and inspecting execution logs, allowing both quick checks and more detailed analysis.

5.4 Validation and Testing

The system was evaluated through input validation, structured test specifications, and end-to-end execution of compatibility checks.

At the translation stage, specifications are checked against a restricted subset of OpenAPI. Inputs that violate structural requirements or fall outside the supported subset are rejected with structured errors, ensuring that only valid representations reach the formal core.

Test specifications are used to exercise the pipeline, including well-formedness fixtures and compatibility pairs ([specs/tests/](#)). These tests include both intentionally compatible evolutions (such as added optional fields or endpoints) and intentionally incompatible changes (such as removed endpoints, missing required fields, or stricter request requirements).

End-to-end behaviour is validated by running these specifications through the full pipeline, from translation and code generation to Agda-based checking. The resulting outputs are inspected to confirm correct classification of compatibility and accurate reporting of drift.

6 Results and Discussion

This project shows that compatibility checking for OpenAPI specifications can be treated as a precise, executable problem. Rather than relying only on structural differencing or heuristic rules, the checker evaluates compatibility against a formally defined refinement relation.

When compatibility does not hold, the system produces a structured drift witness identifying the specific structural change responsible. This makes failures more useful in practice, since the checker does not simply reject a change but indicates why it is incompatible.

The end-to-end pipeline also shows that the formal model can be applied in practice. OpenAPI specifications are translated into the internal DSL, compiled, and checked through the Agda formalisation. Results are then surfaced through backend and frontend tooling, validated across test cases covering compatible evolutions, breaking changes, and ill-formed inputs.

7 Related Work

Existing tools for comparing OpenAPI specifications rely on structural differencing and rule-based heuristics. For example, *openapi-diff* [7] detects changes such as added or removed endpoints, modified parameters, and schema differences. These approaches focus on syntactic comparison rather than formal guarantees.

Prior work on API evolution highlights the difficulty of maintaining backward compatibility as systems evolve. Empirical studies of web APIs show that breaking changes occur in practice and are often introduced without clear documentation or deprecation strategies [1, 8]. Similar patterns have been observed in software libraries, where empirical studies show that the majority of breaking API changes arise from refactorings such as renaming, moving, or splitting classes and methods [9].

Formal approaches to software correctness model safe evolution using refinement and type-based reasoning. In type systems, subtyping and refinement capture when one component can safely replace another without breaking existing behaviour [5]. Similar ideas appear in work on behavioural subtyping and interface refinement [10], though not in the setting of OpenAPI specifications.

This work models API compatibility as a refinement relation over OpenAPI specifications. Unlike existing tools, it provides a formally defined notion of compatibility together with an executable decision procedure that either constructs a proof of compatibility or produces a structured explanation of incompatibility.

8 Conclusion and Future Work

This project developed a compatibility checker for OpenAPI specifications based on a formally defined notion of refinement. The formal core, implemented in Agda, defines well-formedness, refinement, and decidable compatibility checking, while the surrounding pipeline translates real OpenAPI specifications into the internal representation, compiles and executes the checker, and presents results through a web interface.

The project shows that API compatibility can be treated as a decidable formal property rather than a heuristic judgement. When compatibility fails, the checker produces a structured drift witness identifying the specific structural change responsible, making the result both precise and explainable.

8.1 Future Work

The current system focuses on a restricted subset of OpenAPI in order to keep the formal model precise and the checking procedure decidable. A natural next step is extending support to additional OpenAPI features, particularly schema composition constructs such as `oneOf` and `anyOf`, which are common in real specifications but more difficult to model formally.

Another useful extension would be reporting multiple independent drift witnesses in a single run, allowing several breaking changes to be surfaced at once.

There is also scope to improve execution efficiency. The current pipeline recompiles generated Agda modules for each run, which introduces overhead. Reducing compilation costs or reusing compiled artefacts could make the checker more practical for larger specifications or more frequent use.

References

- [1] T. Espinha, A. Zaidman, and H.-G. Gross. Web API growing pains: Stories from client developers and their code. In *2014 Software Evolution Week - IEEE Conference on Software Maintenance, Reengineering, and Reverse Engineering (CSMR-WCRE)*, pages 84–93, 2014. <https://doi.org/10.1109/CSMR-WCRE.2014.6747228>.
- [2] P. M. Cray. OpenAPI Specifications in the Real World. White Paper, APIContext, August 2024. <https://emails.apicontext.com/hubfs/6-%20White%20Papers/APIContext%20API%20Drift%20White%20Paper.pdf>.
- [3] OpenAPI Initiative. OpenAPI Specification. <https://spec.openapis.org/oas/latest.html>.
- [4] U. Norell. Dependently Typed Programming in Agda. In P. W. M. Koopman, R. Plasmeijer, and S. D. Swierstra (eds.), *Advanced Functional Programming*, 6th International School (AFP 2008), Lecture Notes in Computer Science, vol. 5832, pages 230–266. Springer, 2009. https://doi.org/10.1007/978-3-642-04652-0_5.
- [5] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [6] Agda Development Team. Termination Checking. Agda User Manual. <https://github.com/agda/agda/blob/master/doc/user-manual/language/termination-checking.lagda.rst>.
- [7] OpenAPI Tools. openapi-diff: A tool for comparing two OpenAPI specifications. <https://github.com/OpenAPITools/openapi-diff>.
- [8] F. Di Lauro, S. Serbout, and C. Pautasso. Towards Large-Scale Empirical Assessment of Web APIs Evolution. In *Proceedings of the 21st International Conference on Web Engineering (ICWE)*, pages 124–138. Springer, 2021. https://doi.org/10.1007/978-3-030-74296-6_10.
- [9] D. Dig and R. Johnson. How do APIs evolve? A story of refactoring. *Journal of Software Maintenance and Evolution: Research and Practice*, 18(2):83–107, 2006. <https://doi.org/10.1002/smr.328>.
- [10] B. H. Liskov and J. M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, 1994. <https://doi.org/10.1145/197320.197383>.